

The background of the entire page is a photograph of a fishing net on a raft. The net is green and is being pulled up by two blue poles. The raft is made of dark, rectangular floats. The water is dark blue and has small ripples. The sky is blue with some white clouds.

Procedural Rule Pack Guide

Fish Farms

Contents

Create a new Procedural Rule Pack (PRP)	2
Needed files for PRP	2
Manifest	3
Processor	4
Display Mode.....	5
Procedural Rule Pack Actor	7
Outcome	9

Create a new Procedural Rule Pack (PRP)

This is a step-by-step tutorial on how to make a new PRP, in this case, we are going to be making a Fish Farm PRP pack.

The Fishing Farm PRP will spawn a 3D model in specific locations, these locations being driven by the JSON data.

First, we will create our new folder containing all our new 3D models, materials, textures...

Content/PRPPacks/FishingFarms

This is how you should manage your assets inside your new PRP folder

- 00_Assets
 - 00_Blueprints
 - 01_Meshes
 - 02_Materials
 - 03_Textures
- 01_DisplayModes
- 02_Processors
- 03_Manifest

Needed files for PRP

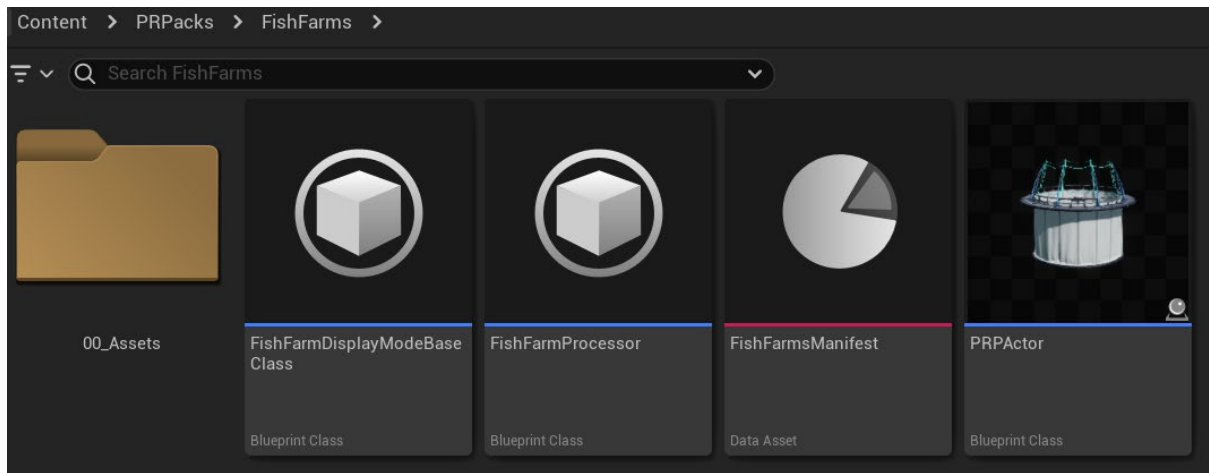
What do we need?

1. **Manifest:** tells the game what processors and display modes to execute for this PRP
 - 1.1. **Processors:** Allows us to edit the JSON data and save it in a new data pool layer
 - 1.1.1. **Display Modes:** Gets a data pool layer and uses it to spawn our desired actors or systems
 - 1.1.1.1. **PRP Actor:** Base class for actors that form part of a display mode in which the data layer can be enabled/disabled through the game UI

How do we make this in Unreal?

In the content browser:

1. Create the Manifest: Right Click → Miscellaneous → Data Asset, and select "Procedural Rule Pack Manifest" in the folder.
2. Create the Processor: Right Click → Blueprint Class, and find "Procedural Rule Pack Data Source Processor"
3. Create the Display Mode: Right Click → Blueprint Class, and find "Procedural Rule Pack Display Mode"
4. Create PRP Actor: Right Click → Blueprint Class and find "Procedural Rule Pack Actor"



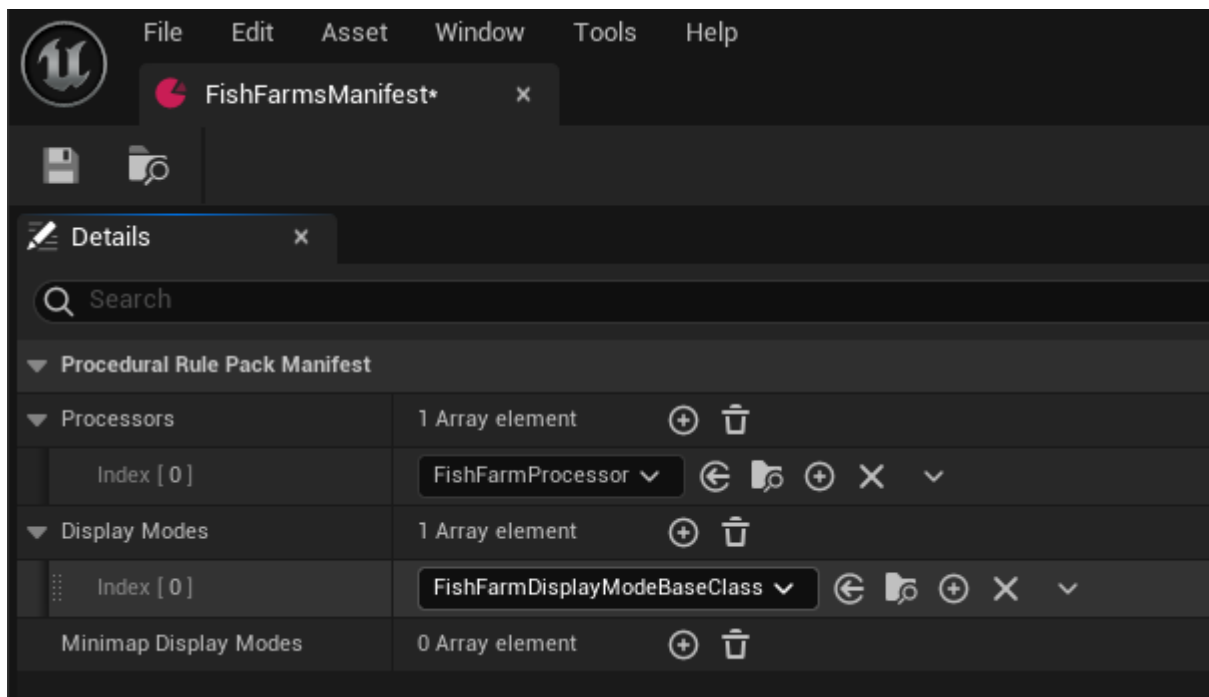
Your PRP folder should look like this now

Manifest

How to let the game know what to execute when it starts?

The game will look for any folder containing a manifest, and run all the Processors and Display Modes specified in it, by order.

So open your manifest and add your newly created Manifest and Processor.



This is how your manifest should look like.

Now our processors and Display Modes will be Executed when we launch the game, in the specified order.

Processor

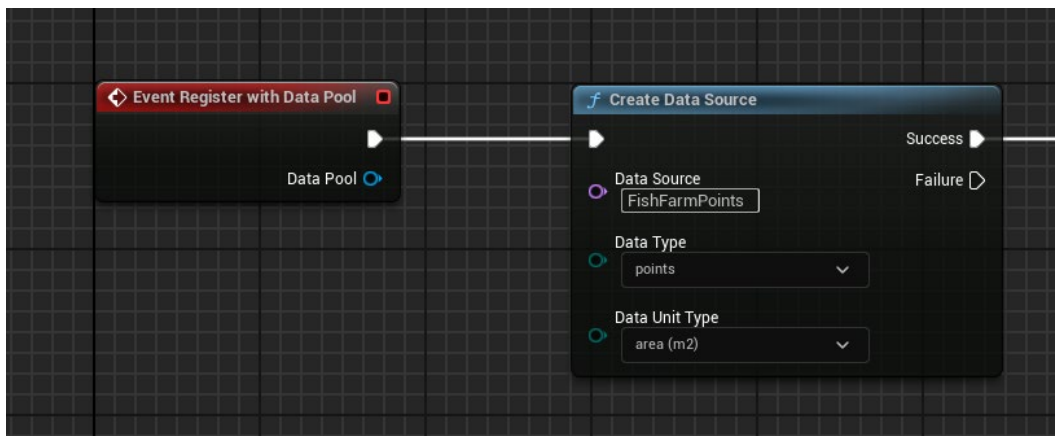
For our example, we are not going to do much in the processor, since we don't need to manipulate the JSON data.

We just need the raw "Fish Farm" point data. If we need to edit the raw data, it would happen in the Processor, since it allows us to store the modified data in another Data Pool Layer with a new desired name.

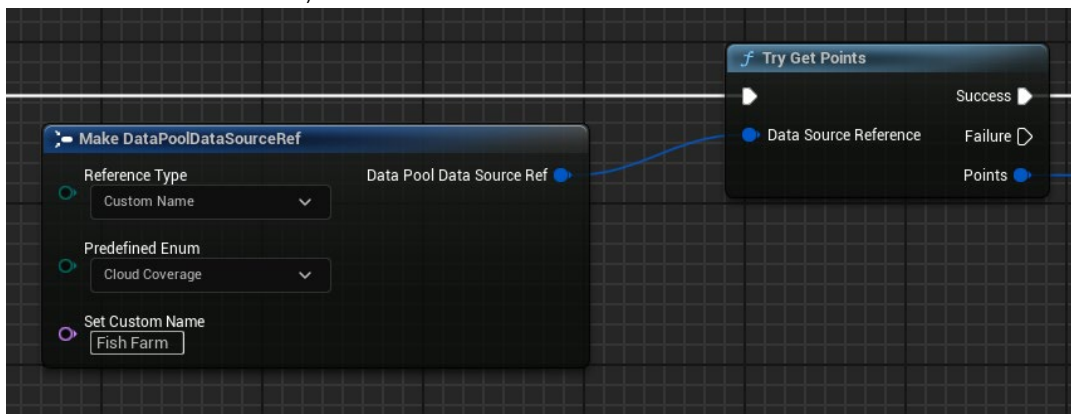
What would our Processor look like?

Our processor contains one function called "Event Register with Data Pool". This gets executed if our Processor was added to the Manifest (explained in [Manifest](#)), so all our data processing needs to be executed after this event.

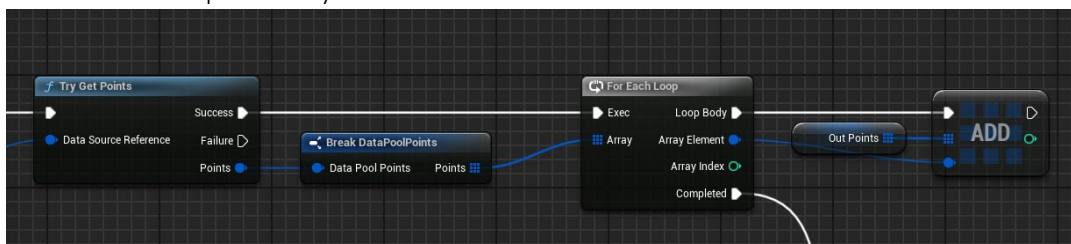
- 1- We create a new Data Source of type points, named "FishFarmPoints", to store our edited data



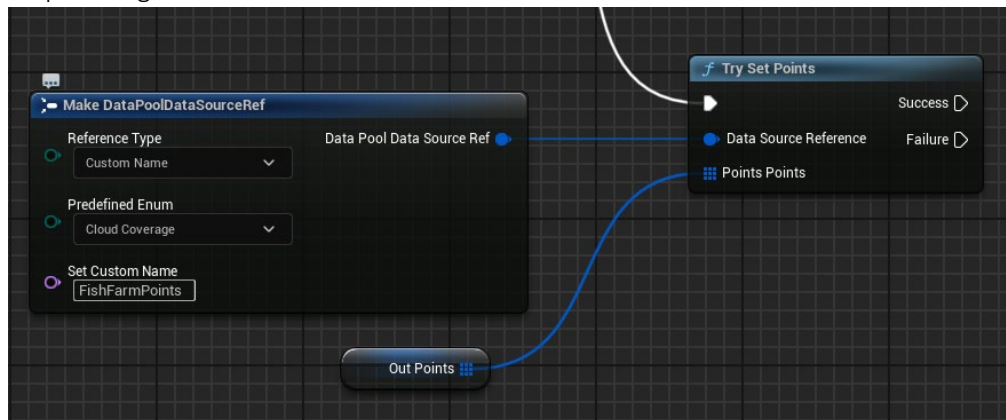
- 2- We try to get the points from the JSON original data layer named "Fish Farm" (this data layer should be in the JSON file)



- 3- After we get the points we can do anything with it, in this case, we are just going to store them into a new point array.



- 4- Once we are happy with our points we can store them in our new Data Layer generated in Step 1 using the same name “FishFarmPoints”.



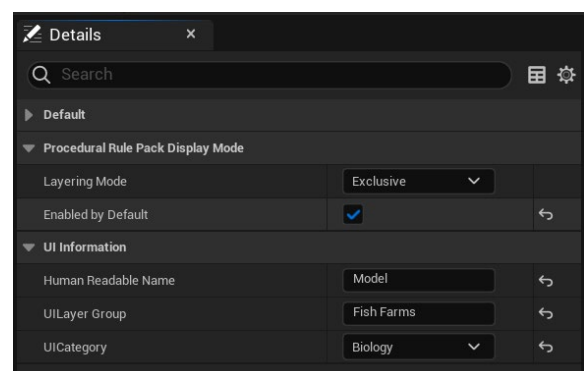
Our processor is done, this will be executed by the manifest and will generate a new Data Source called “FishFarmPoints” containing point data that can be used later.

Display Mode

This Blueprint Class has some default properties that can be edited.

Here you can define:

- Its name
- If it's enabled by default (visibility on/off)
- What UI group/category it belongs to



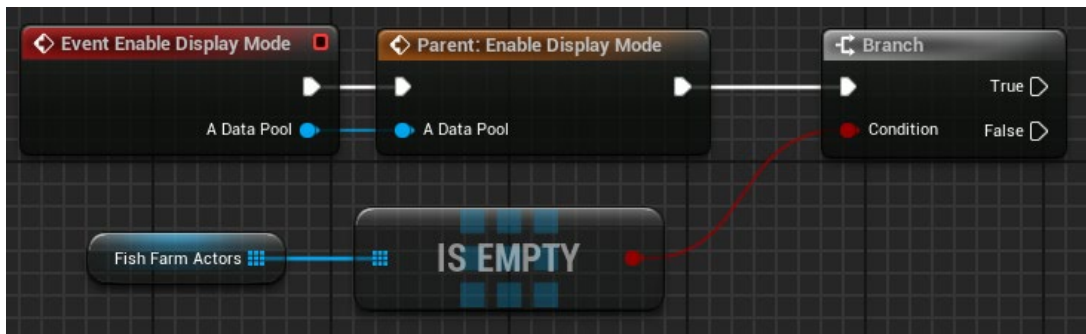
After creating our “FishFarmPoints” Data layer generated by our processor. We will use this in our Display Mode to spawn on every point our 3D model.

In the Display Mode, you find two functions (Event Enable/Disable Display Mode), with it, you can toggle the layer visibility inside of the game, through a dynamically generated UI, based on the class defaults.

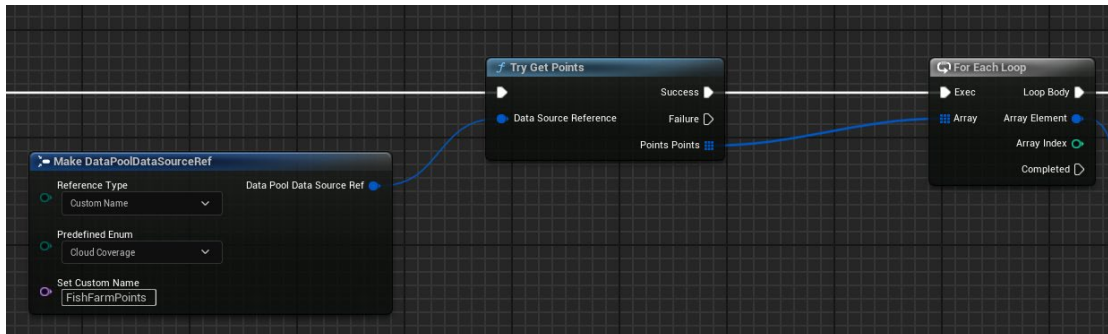
How do we spawn/generate our Fish Farm?

First, we create a system to determine if our actors have already been generated or not, if not, we can initialize our PRP Actor with its new properties. Using the new Point Data we stored in the “FishFarmPoints” Data Layer as the new spawn locations.

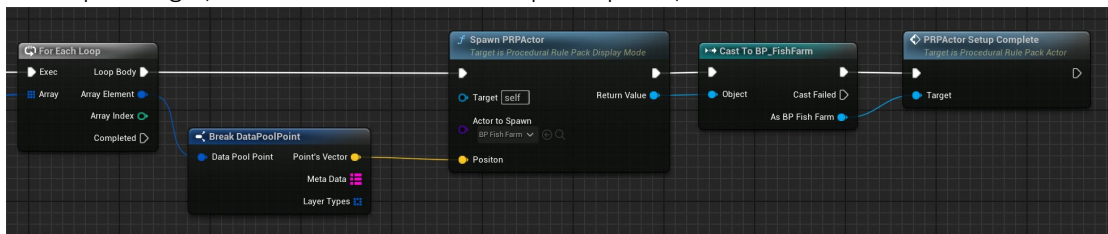
1. Create a Procedural Rule Pack Actor variable of type array, that will store our spawned actors
2. After the event “Event Enable Display Mode” check if our array is empty



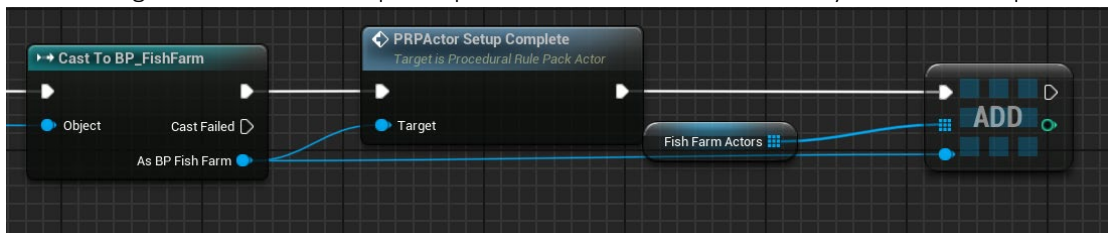
3. If empty we can get our “FishFarmPoints” data and loop through the points



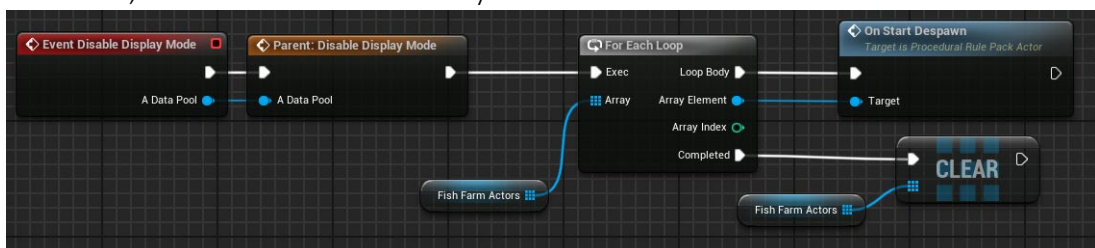
4. When looping through each point we are extracting the position, and using their value we will spawn our PRPActor at the desired location.
After spawning it, we call the “PRPActor Setup Complete”, to initialize the actor.



5. After calling our “PRPActor Setup Complete” we will store it in our array created in Step 2



6. To make sure our spawned actors despawn when we toggle the UI, through the “Event Disable Display mode”, we need to call for every PRP Actor in our array → “On Start Despawn”.
At the end, we clear the PRP Actors array.

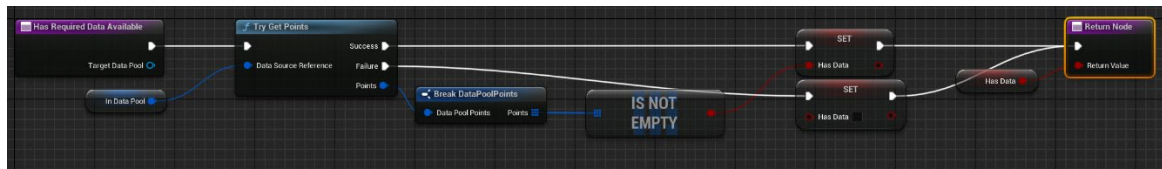


Now we have a fully functional display mode that spawns in our Fish Farm PRP Actor based on the newly created data layer.

What if our JSON Config file doesn't carry the data layer we need?

Our Display Mode has an inherited function that's called "Has Required Data Available", here we can set up a system to determine if we have enough data to generate the Display Mode. If set to True the display mode will show up in the game, if set to False, the display Mode will be disabled and wont show up in the UI.

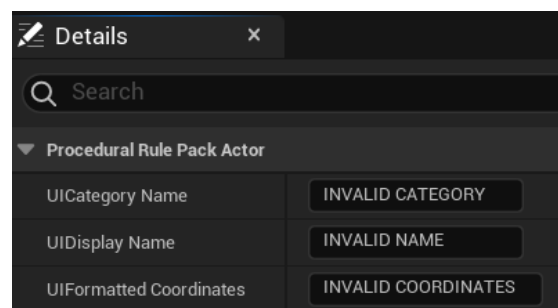
In our case, we check if our Data Layer contains no Point data, if so we toggle the Boolean accordingly:



Procedural Rule Pack Actor

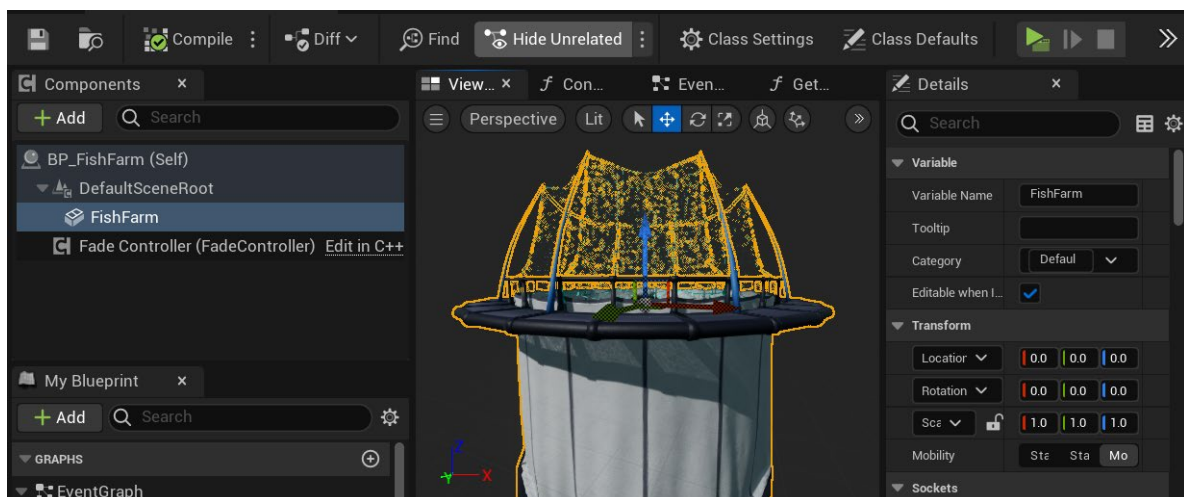
This Blueprint Class contains inherited parameters and events found in the Class Defaults:

- UI Name
- UI Group
- And its UI Coordinates



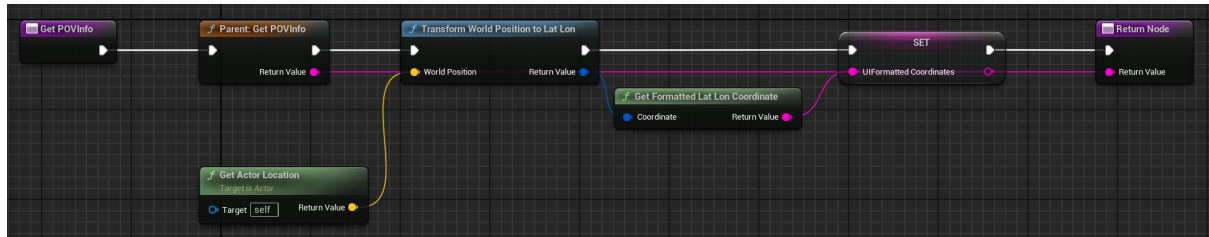
These will show up in a UI when the Actor is selected.

This actor contains everything that we want to spawn, Static Meshes, lights, systems... In this case, to set up our Fish Farm we add a Static Mesh Component:



For this we want to change the coordinates properties of our asset based on its location in the world, to see what the corresponding coordinates are for the selected asset.

To do this we can use one of the inherited functions "get POVInfo", and through it, set our "UIFormatted Coordinates" parameter like this:



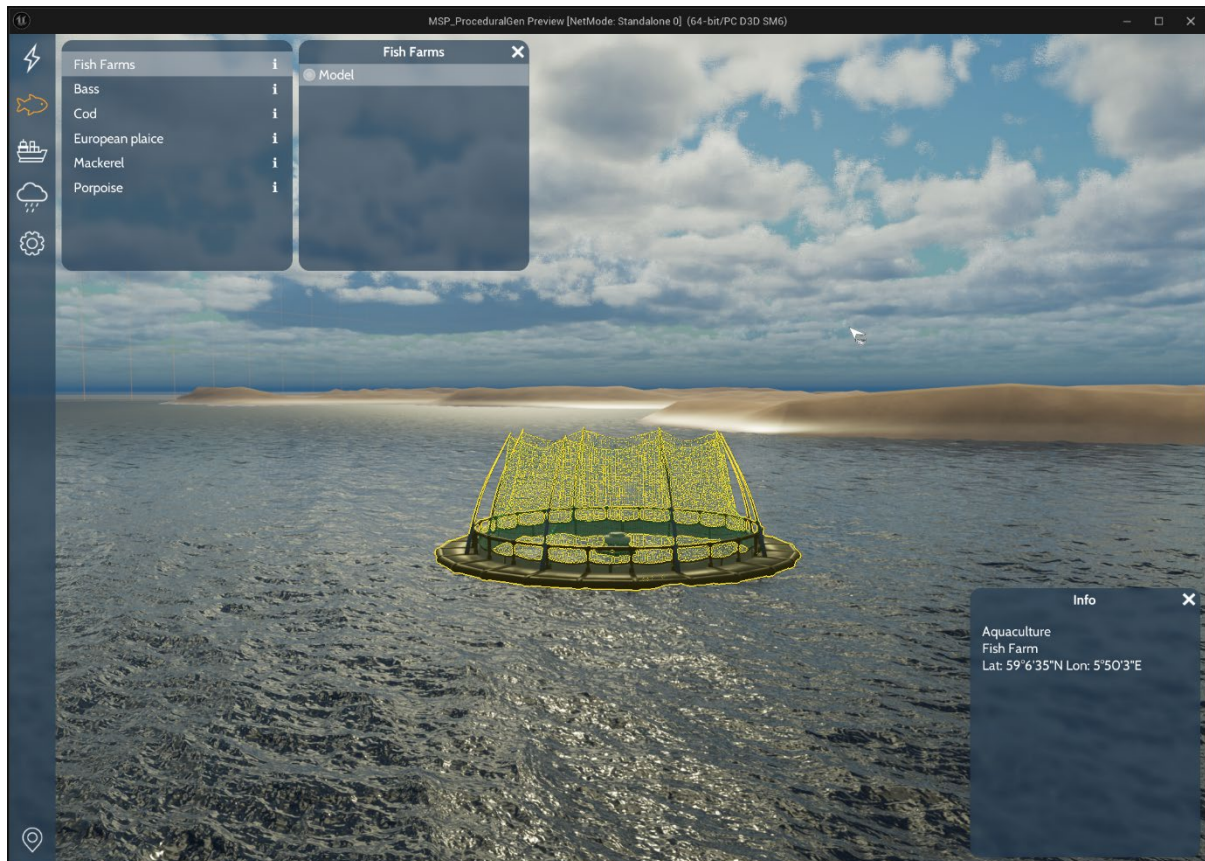
This is an example, but the PRP Actor can have its blueprint functionalities, to determine this actor's behavior in the world.

To initialize any behavior or functions, use the “PRP Actor Setup Complete” event, since this one gets called from the Display mode when the actor gets spawned if set up correctly (shown in the [Display Mode](#) section).

Outcome

After creating all the components, this is what we will get when we launch the game:

- PRP Actor placed in the world
- Dynamic UI generated allowing to toggle visibility
- UI Information box with dynamic coordinates system and asset properties.



If your asset and UI don't show up, likely, that the JSON Config file you are loading doesn't contain the "Fish Farm" data layer. So it gets disabled.